

Window Function

Advanced Databases

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence

Bases and coordinates

1. Motivation - The ”Why”
2. Window Function
3. Main Categories of Window Functions

Learning Objectives

By the end of this session, you will be able to:

1. **Define** the core difference between Aggregate Functions (‘GROUP BY‘) and Analytic Functions (‘Window‘).
2. **Construct** complex queries using ‘OVER‘, ‘PARTITION BY‘, and ‘ORDER BY‘.
3. **Analyze** time-series data using Sliding Windows and Value functions (‘LAG‘, ‘LEAD‘).
4. **Avoid** common pitfalls regarding the SQL Order of Execution (Why you can’t use Window Functions in ‘WHERE‘).
5. **Apply** ranking techniques for ”Top-N” problems in Recommender Systems.

Motivation - The ”Why”

Motivation: Why Learn Window Functions?

The Question: Why not just use standard $\mathrm{GROUP\ BY}$ or Subqueries?

The Pain (Old SQL)

1. **Complexity:** Comparing a row to a group average requires complex **Self-Joins** or nested subqueries.
2. **Performance:** calculating Running Totals with subqueries is often $O(N^2)$. It kills performance on Big Data.
3. **Limitations:** Solving "Top 3 items per Category" is extremely difficult and verbose with standard SQL.

The Power (Window Functions)

1. **Simplicity:** Clean, readable syntax using $\mathrm{OVER}()$.
2. **Efficiency:** The database processes the data in a **single pass** (plus sorting). Typical complexity: $O(N \log N)$.
3. **AI Relevance:** Essential for **Time-Series Feature Engineering** (Lags, Rolling Averages, Trends).

Real-World Problems Solved by Window Functions

What specific issues do they address?

1. **The "Top-N" Problem (Ranking):**
   - *Scenario:* "Find the top 3 highest-paid employees in *each* department."
   - *Solution:* ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC).
2. **The "Moving Target" Problem (Time Series):**
   - *Scenario:* "Calculate the 7-day moving average of stock prices."
   - *Solution:* AVG(price) OVER (... ROWS BETWEEN 6 PRECEDING AND CURRENT ROW).
3. **The "Gap & Island" Problem (Advanced):**
   - *Scenario:* Identifying periods of continuous activity vs. gaps in user sessions.
   - *Solution:* Using LAG() and LEAD() to detect state changes.

Case Study: Customer Behavior Analysis

1. The Business Requirement

- The Marketing Team wants to understand customer retention.
- **Question:** *"What is the average waiting time (days) before a customer rents another movie?"*

2. The Technical Challenge (Feature Engineering) To answer this, for **every single rental transaction**, we must:

1. Identify the **Current Rental Date** (t).
2. Find the **Previous Rental Date** ($t-1$) for the *same customer*.
3. Calculate the interval:

$$\Delta t = \text{Current Date} - \text{Previous Date}$$

Goal: Create a new feature column $\mathrm{days_diff}$ *to train a Churn Prediction Model.*

Window Function

What is a Window Function?

Standard Definition

A Window Function performs a calculation across a set of table rows that are somehow related to the current row.

Crucial Distinction:

It does **NOT** cause rows to become grouped into a single output row like $\mathrm{GROUP\ BY}$.

In short: Calculation ”looking around” the current row.

The Key Comparison: GROUP BY vs. Window

GROUP BY (Aggregation)

- Collapses multiple rows $\rightarrow$ 1 row.
- **Loses** individual payment details.

```
SELECT customer_id, SUM(amount)
FROM payment
GROUP BY customer_id;

-- Result: 1 row per customer.
-- (We lose the date/amount of specific
    payments)

```

Window Function

- Does **not** collapse rows.
- Adds "Total per Customer" to each payment row.

```
SELECT payment_id, customer_id, amount,
 SUM(amount) OVER (
    PARTITION BY customer_id
 ) AS total_spent_by_cust
FROM payment;

-- Result: Keeps all payment rows.
-- (Can compare single payment vs. total)

```

Anatomy of the ”Window”

What defines the ”Window” (The set of rows the function sees)?

```
OVER (
  PARTITION BY ...   -- (1) Scoping
  ORDER BY ...       -- (2) Sequence
  ROWS / RANGE ...   -- (3) Framing
)
```

1. **PARTITION BY:** Divides rows into logical groups (buckets). The function resets for each partition.
2. **ORDER BY:** Defines the sequence *within* the partition (Critical for Running Totals).
3. **Frame Clause:** Defines the physical range relative to the current row (e.g., ”Previous 2 rows”).

Visualizing ”Looking Around” (Sliding Window)

Scenario: Calculate the sum of the last 3 payments for each customer.

```
SELECT payment_id, amount,
  SUM(amount) OVER (
    PARTITION BY customer_id
    ORDER BY payment_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) as rolling_3_payments
FROM payment;
```

How it works:

- For the **Current Payment**:
- Look at **2 payments before** it + **Current payment**.
- Sum the amount.
- Move to the next payment and repeat (Slide).

Main Categories of Window Functions

Main Categories of Window Functions

1. Ranking Assigns an integer rank.

- ROW_NUMBER()
- RANK() (Skips ties)
- DENSE_RANK() (No skips)

Use case: Top-N Leaders.

2. Aggregate Standard agg functions.

- SUM(), AVG()
- COUNT()
- MIN(), MAX()

Use case: Running Totals.

3. Value (Offset) Access other rows.

- LAG() (Previous)
- LEAD() (Next)
- FIRST_VALUE()

Use case: Growth Rate.

Ranking Functions: Handling Ties

Concept: Assigning a specific order integer to rows.

- **ROW_NUMBER():** Unique sequence. Ties are broken arbitrarily (non-deterministic) unless you add more columns to ORDER BY.
- **RANK():** Ties get the same rank. Skips subsequent numbers (leaves gaps).
- **DENSE_RANK():** Ties get the same rank. No gaps in numbering.

Visual Comparison (Imagine 3 films with the same length):

Film	Length	ROW_NUMBER	RANK	DENSE_RANK
Film A	120 min	1	1	1
Film B	120 min	2	1	1
Film C	110 min	3	**3**	**2**

```sql
-- Practical: Find the top longest films
SELECT title, length,
  DENSE_RANK() OVER (ORDER BY length DESC) as length_rank
FROM film LIMIT 5;

```

Aggregate Window Functions

Concept: Apply standard aggregates (SUM, AVG, COUNT) without collapsing rows.
Two main modes of operation:

A. Group Metric (Static)

- Uses PARTITION BY only.
- Result: Same value for every row in the group.
- *Ex: Total spent by customer.*

B. Running Metric (Cumulative)

- Uses PARTITION BY + ORDER BY.
- Result: Cumulative calculation up to current row.
- *Ex: Running total of spending.*

```
SELECT customer_id, payment_date, amount,
  -- Cumulative Sum (Running Total)
  SUM(amount) OVER (
      PARTITION BY customer_id
      ORDER BY payment_date
  ) as running_total
FROM payment;
```

Value (Offset) Functions

Concept: Access data from a row physically located *N* positions away from the current row.

- **LAG(col, n):** Look *backwards n* rows.
- **LEAD(col, n):** Look *forwards n* rows.
- **FIRST_VALUE / LAST_VALUE:** Get the very first or last item in the window.

```
SELECT customer_id, payment_date, amount,
  -- Get previous payment amount
  LAG(amount, 1) OVER (
      PARTITION BY customer_id ORDER BY payment_date
  ) as prev_amount,

  -- Calculate Delta
  amount - LAG(amount, 1) OVER (
      PARTITION BY customer_id ORDER BY payment_date
  ) as diff_from_last_time
FROM payment;

```

Logical Order of Execution (Critical)

Window Functions are calculated **AFTER**: FROM, WHERE, GROUP BY, HAVING.
But **BEFORE**: ORDER BY (Final sort).

Common Mistake

```
SELECT * FROM employees
WHERE ROW_NUMBER() OVER(ORDER BY salary) = 1;   -- ERROR!
```

Reason: You cannot filter by a Window Function in the WHERE clause because it hasn't been calculated yet.

Solution (Use CTE or Subquery):

```
WITH RankedEmp AS (
  SELECT *, ROW_NUMBER() OVER(ORDER BY salary) as rn FROM employees
)
SELECT * FROM RankedEmp WHERE rn = 1;
```

The Complete Execution Pipeline

Memorize this sequence to debug complex queries:

1. FROM / JOIN *(Load tables, build Cartesian product)*
2. WHERE *(Filter raw rows)*
3. GROUP BY *(Collapse rows into groups)*
4. HAVING *(Filter groups)*
5. WINDOW FUNCTIONS ← **HERE!**
6. SELECT *(Select columns / Projection)*
7. DISTINCT *(Remove duplicates)*
8. ORDER BY *(Sort final result)*
9. LIMIT / OFFSET *(Cut result)*

Key Takeaway: Window Functions run **after** filtering/grouping but **before** the final sort/limit.

Why this specific position?

A. After WHERE / GROUP BY: The Window Function only sees the "Survivors".

```
SELECT emp_name, salary,
       RANK() OVER (ORDER BY salary) -- (2) Then Rank
FROM employees
WHERE salary > 1000;                  -- (1) Filter first
-- Result: Employees with salary <= 1000 are invisible to RANK()

```

B. Before ORDER BY: The Window Function calculates based on its *own* rules, independent of the final presentation.

```
SELECT emp_name, salary,
       -- Rank by Salary High -> Low
       ROW_NUMBER() OVER (ORDER BY salary DESC) as rn
FROM employees
ORDER BY emp_name ASC; -- Final Output sorted by Name
-- Result: "rn" is correct based on salary, even if rows are mixed.

```

What Window Functions are NOT

To understand the tool, you must understand its limits.

They are NOT:

- A Subquery.
- A JOIN operation.
- A replacement for Indexing.

They do NOT handle:

- Concurrency Control (Locking).
- ACID Properties.
- Data Modification (INSERT/UPDATE).

Core Domain: They are strictly for **Read-Only Data Analytics**.

Lab 1: Film Ranking (The Basics)

Business Scenario: The Content Team wants to schedule a movie marathon. They need the top 3 longest films within **each Category**.

Task:

1. Join film, $\mathrm{film_category}$, and $\mathrm{category}$ tables.
2. Use $\mathrm{DENSE_RANK}()$ to rank film lengths (descending) per category.
3. *Hint: Use PARTITION BY category name.*

Lab 2: Growth Analysis (Time Series)

AI Context: For Sales Forecasting models, raw revenue is not enough. We need the **Month-over-Month (MoM) Growth Rate** as a feature.

Task:

1. Create a CTE to calculate total revenue per month ($\mathrm{DATE_TRUNC}$).
2. Use $\mathrm{LAG}()$ to retrieve the previous month's revenue.
3. Calculate Growth %: $\frac{\text{Current} - \text{Previous}}{\text{Previous}} \times 100$.

Lab 3: Customer Lifetime Value (Running Total)

AI Context: We need to track the *Customer Lifetime Value (CLV)* at the exact moment of each transaction to predict Churn.

Task:

1. Use the payment table.
2. Calculate the **Cumulative Sum** of the amount column for each customer.
3. Order the data chronologically by payment_date.

Lab 4: The ”Top-N” Filter Challenge

The Challenge: Find the single highest-spending customer (Top 1) for **each Country**.
Problem: You cannot use $\mathrm{WHERE\ rank} = 1$ *directly in the SELECT statement.*
Instructions:

1. **Step 1:** Join 4 tables ($\mathrm{customer}$, $\mathrm{address}$, city, $\mathrm{country}$) to aggregate total spend by Customer and Country.
2. **Step 2:** Use a CTE to apply $\mathrm{RANK}()$ partitioned by Country.
3. **Step 3:** Select from the CTE where $\mathrm{rank} = 1$.